

Unidad Didáctica

**Introducción a la Programación,
Unidad I**

Contenido

Introducción.....	3
Desarrollo de Contenidos	4
Máquinas y programas	4
Programación e ingeniería de software.....	5
Lenguajes de programación	5
Evolución de los lenguajes de programación.....	7
Lenguaje de máquina	7
Lenguaje de Bajo nivel.....	8
Lenguaje de Alto nivel.....	9
Compiladores e intérpretes.....	9
Conceptos Básico de la programación	10
Tipos de Errores	10
Verificación y Depuración	11
Documentación y Mantenimiento	12
Programas	13
Partes de un Programa	15
Elementos de la programación imperativa	16
Fases de Desarrollo de un Programa	17
Fundamentos del entorno de desarrollo del lenguaje de programación.....	18
Bibliografía y Webgrafía	20
Glosario	20

Introducción



La programación es una habilidad esencial en la era digital en la que vivimos, ya que es la base para el desarrollo de software y aplicaciones que utilizamos diariamente. En esta

unidad introductoria, exploraremos los conceptos fundamentales de la programación, desde la evolución de los lenguajes de programación hasta los elementos de la programación imperativa.

Comenzaremos por analizar las máquinas y programas, entendiendo cómo la programación e ingeniería de software se han convertido en un componente vital para el éxito empresarial en la actualidad. Además, exploraremos los diferentes lenguajes de programación que existen y cómo han evolucionado desde los lenguajes de máquina y bajo nivel hasta los lenguajes de alto nivel.

También abordaremos los conceptos básicos de la programación, incluyendo los tipos de errores que pueden ocurrir al programar, así como las técnicas de verificación y depuración que se utilizan para solucionarlos. Analizaremos la documentación y el mantenimiento de los programas, así como las partes que los componen. Además, discutiremos las fases de desarrollo de un programa y los fundamentos del entorno de desarrollo del lenguaje de programación

Máquinas y programas

La relación entre máquinas y programas es esencial en el mundo de la informática y la programación. Las máquinas son dispositivos electrónicos que pueden realizar tareas según su diseño y programación, mientras que los programas son conjuntos de instrucciones y datos que se utilizan para controlar el comportamiento de las máquinas.

En términos simples, una máquina es como un "cerebro" que puede procesar información, realizar cálculos y ejecutar acciones. Sin embargo, una máquina no puede realizar estas tareas por sí sola, requiere de un programa que le indique qué hacer y cómo hacerlo. Es decir, una máquina necesita ser programada para ser útil.

Los programas se crean mediante lenguajes de programación, que permiten a los programadores escribir código en un formato legible por humanos que luego se traduce a un lenguaje que la máquina pueda entender. Una vez que se ha escrito el programa, se carga en la máquina y se ejecuta.

En resumen, las máquinas y los programas están intrínsecamente relacionados en la informática. Las máquinas son dispositivos que necesitan ser programados para ser útiles, y los programas son conjuntos de instrucciones que permiten a las máquinas procesar información, realizar tareas y ejecutar acciones.

Programación e ingeniería de software

La programación y la ingeniería de software están estrechamente relacionadas porque la programación es una actividad central en la ingeniería de software. La programación es el proceso de crear código fuente para un programa de software, mientras que la ingeniería de software es el proceso de diseñar, crear, probar y mantener software de calidad.

La programación es solo una parte de la ingeniería de software, ya que también incluye la planificación, el diseño, la documentación, el análisis y la gestión del proyecto. La ingeniería de software se enfoca en el proceso completo de desarrollo de software, mientras que la programación se enfoca en la creación del código fuente.

La programación y la ingeniería de software trabajan juntas para producir software de calidad que cumpla con las necesidades del usuario y sea eficiente en términos de costos y tiempo. La programación es la herramienta que los ingenieros de software utilizan para crear los componentes del software y construir el sistema final. La ingeniería de software, por otro lado, es el enfoque holístico que asegura que el software esté bien diseñado, se construya adecuadamente, se pruebe y se mantenga correctamente. Juntos, la programación y la ingeniería de software son fundamentales para el éxito de un proyecto de software.

Lenguajes de programación

Un lenguaje de programación es un conjunto de símbolos, reglas y estructuras que permiten a los programadores comunicar instrucciones a una computadora u otro tipo de dispositivo. Estos lenguajes permiten a los humanos comunicar tareas y algoritmos de una manera que la computadora pueda entender y ejecutar.

Los lenguajes de programación se dividen en diferentes tipos, que se utilizan para diferentes tipos de tareas y aplicaciones. Algunos de los tipos de lenguajes de programación incluyen lenguajes de programación de bajo nivel, lenguajes de programación de alto nivel y lenguajes de scripting.

Los lenguajes de programación se utilizan en una variedad de industrias y aplicaciones, desde la creación de aplicaciones web y móviles hasta el desarrollo de software de sistemas y el análisis de datos. Cada lenguaje de programación tiene sus propias fortalezas y debilidades, lo que significa que algunos lenguajes son mejores para ciertas tareas que otros.

Ejemplos:

- C: Lenguaje de programación de propósito general ampliamente utilizado. Es conocido por su eficiencia y su capacidad para realizar tareas de bajo nivel.
- Python: Lenguaje de programación interpretado de alto nivel utilizado para una amplia variedad de aplicaciones, como análisis de datos, aprendizaje automático, desarrollo web y automatización de tareas.
- Java: Lenguaje de programación de propósito general que se utiliza principalmente para el desarrollo de aplicaciones empresariales y de escritorio. Se ejecuta en la plataforma Java, lo que lo hace altamente portátil y compatible con diferentes sistemas operativos.
- JavaScript: Lenguaje de programación interpretado utilizado principalmente para el desarrollo web. Permite la creación de interacciones y animaciones en la página web y es uno de los lenguajes más populares en la programación web.
- SQL: Lenguaje de programación utilizado para gestionar y consultar bases de datos relacionales. Permite la creación, modificación y eliminación de tablas y registros en una base de datos.

- PHP: Lenguaje de programación de servidor utilizado principalmente para el desarrollo web. Se utiliza para crear aplicaciones web dinámicas, como sitios web de comercio electrónico y sistemas de gestión de contenidos.
- Ruby: Lenguaje de programación interpretado utilizado principalmente para el desarrollo web y aplicaciones móviles. Es conocido por su simplicidad y facilidad de uso.
- Swift: Lenguaje de programación desarrollado por Apple para el desarrollo de aplicaciones para sus dispositivos iOS, macOS y watchOS. Se basa en el concepto de seguridad y eficiencia.
- C#: Lenguaje de programación de propósito general desarrollado por Microsoft. Se utiliza principalmente para el desarrollo de aplicaciones de escritorio y de Windows, así como para el desarrollo de videojuegos

Evolución de los lenguajes de programación

Lenguaje de máquina

El lenguaje de máquina es el lenguaje que entienden los ordenadores, y está formado por instrucciones binarias, es decir, secuencias de ceros y unos que representan operaciones muy básicas que se realizan directamente en el hardware del ordenador. Estas instrucciones son muy difíciles de entender para los humanos, ya que están compuestas por una serie de números binarios que corresponden a operaciones de bajo nivel.

Cada tipo de procesador tiene su propio conjunto de instrucciones de máquina, y el lenguaje de máquina es específico para cada tipo de hardware. Es por ello que resulta complicado programar en lenguaje de máquina directamente, ya que es muy laborioso y propenso a errores.

A pesar de su complejidad, el lenguaje de máquina es la forma en que los ordenadores ejecutan cualquier tipo de programa, ya que todas las instrucciones de

los lenguajes de programación más avanzados son finalmente convertidos en instrucciones de máquina antes de su ejecución en el procesador. Los programas compiladores son los encargados de traducir el código fuente escrito en un lenguaje de programación a lenguaje de máquina para que pueda ser ejecutado.

Lenguaje de Bajo nivel

El lenguaje de bajo nivel es un tipo de lenguaje de programación que se encuentra más cerca del lenguaje de máquina. Está diseñado para interactuar directamente con el hardware del sistema, es decir, con la CPU, la memoria y los dispositivos de entrada/salida. Los lenguajes de bajo nivel son muy precisos y eficientes, pero también son muy difíciles de entender y utilizar.

El lenguaje de bajo nivel utiliza un conjunto limitado de instrucciones básicas para comunicarse con el hardware del sistema. A menudo se escribe en código hexadecimal o en código ensamblador, lo que significa que las instrucciones se representan mediante códigos de dos o tres letras en lugar de palabras completas. Cada instrucción de bajo nivel se corresponde con una sola operación en la CPU, lo que significa que los programas escritos en lenguaje de bajo nivel pueden ejecutarse muy rápidamente.

Algunos ejemplos de lenguajes de bajo nivel incluyen el lenguaje ensamblador de la CPU de Intel (x86), el lenguaje ensamblador de la CPU de ARM y el lenguaje de programación C. Aunque el lenguaje de programación C se considera un lenguaje de alto nivel, es un lenguaje de bajo nivel en comparación con otros lenguajes de programación de alto nivel, como Java o Python.

Lenguaje de Alto nivel

Los lenguajes de programación de alto nivel son aquellos que tienen un alto nivel de abstracción, es decir, utilizan una sintaxis más cercana al lenguaje natural y menos cercana al lenguaje de máquina. Estos lenguajes permiten a los programadores escribir código de forma más rápida y sencilla, ya que se enfocan en la solución de problemas en lugar de preocuparse por la arquitectura de la máquina.

Los lenguajes de alto nivel se clasifican por lo general en dos categorías: los lenguajes interpretados y los lenguajes compilados. Los lenguajes interpretados son aquellos cuyo código fuente se ejecuta directamente en un intérprete, mientras que los lenguajes compilados se traducen en código de máquina antes de la ejecución.

Algunos ejemplos de lenguajes de programación de alto nivel son Python, Java, Ruby, C#, JavaScript, entre otros. Estos lenguajes ofrecen una gran variedad de bibliotecas y frameworks que permiten a los programadores crear aplicaciones web, móviles, juegos, entre otros. Además, son lenguajes más fáciles de aprender y utilizar que los lenguajes de bajo nivel, por lo que son ideales para programadores novatos y para el desarrollo rápido de prototipos.

Compiladores e intérpretes

Los compiladores e intérpretes son dos tipos de programas que se utilizan en el desarrollo de software. Ambos son herramientas que traducen código fuente escrito por un programador en lenguaje de máquina, que es el lenguaje que entiende el hardware de una computadora.

Un compilador es un programa que traduce todo el código fuente escrito por el programador a lenguaje de máquina en un solo paso. Es decir, el compilador toma el código fuente completo y lo procesa para generar un archivo ejecutable, que se puede ejecutar en una computadora sin necesidad de tener el código fuente original.

Por otro lado, un intérprete es un programa que ejecuta el código fuente escrito por el programador línea por línea. Es decir, el intérprete lee el código fuente y lo traduce a lenguaje de máquina a medida que se va ejecutando. Esto significa que el intérprete no genera un archivo ejecutable, sino que interpreta el código fuente en tiempo real.

En resumen, la principal diferencia entre un compilador y un intérprete es que un compilador traduce todo el código fuente de una sola vez y genera un archivo ejecutable, mientras que un intérprete traduce el código fuente línea por línea en tiempo real.

Conceptos Básico de la programación

Tipos de Errores

En programación, un error es una falla en el código que impide que el programa se ejecute correctamente. Los errores se dividen en tres categorías principales: errores de sintaxis, errores de tiempo de ejecución y errores lógicos.

1. Errores de sintaxis: estos son errores que ocurren cuando el código no se escribe correctamente y no sigue las reglas de sintaxis del lenguaje de programación. Los errores de sintaxis son fáciles de identificar porque el compilador o intérprete emitirá un mensaje de error. Ejemplos de errores de sintaxis incluyen olvidar cerrar un paréntesis o un corchete, utilizar una palabra clave de manera incorrecta, o escribir mal el nombre de una variable.
2. Errores de tiempo de ejecución: estos errores ocurren cuando el programa se está ejecutando y algo inesperado sucede, como una división por cero, una operación en una variable que no está inicializada o un archivo que no se puede encontrar. Estos errores no se detectan hasta que el programa se está

ejecutando y pueden causar que el programa se detenga o produzca resultados incorrectos.

3. Errores lógicos: estos son errores que no se detectan mediante la verificación de la sintaxis del código o al ejecutar el programa, pero que hacen que el programa no produzca los resultados esperados. Estos errores pueden ser difíciles de encontrar, ya que el código se compila sin errores y el programa se ejecuta sin interrupciones. Ejemplos de errores lógicos incluyen utilizar una condición incorrecta en una estructura de control, utilizar una fórmula incorrecta en un cálculo, o hacer una suposición incorrecta acerca de cómo funciona el programa.

Verificación y Depuración

La verificación y depuración son procesos importantes en la programación que se realizan para garantizar que el código de un programa funcione correctamente y sin errores.

La verificación implica revisar el código en busca de errores antes de ejecutar el programa. Esto se hace mediante pruebas y revisiones cuidadosas del código para asegurarse de que las funciones se comporten según lo previsto y que no haya errores sintácticos o semánticos. La verificación es importante porque cuanto antes se detecten los errores, más fácil es corregirlos antes de que se conviertan en problemas más graves.

La depuración se lleva a cabo después de que se haya detectado un error. El objetivo es encontrar y corregir el error para que el programa funcione correctamente. La depuración puede ser un proceso complejo y requiere el uso de herramientas especiales, como depuradores, que permiten al programador ejecutar el programa

paso a paso y examinar el estado de las variables y las estructuras de datos en tiempo de ejecución.

En general, la verificación y depuración son procesos continuos en la programación, y es importante dedicar tiempo y esfuerzo a ambos para garantizar que el código sea lo más libre de errores posible y se ejecute sin problemas.

Documentación y Mantenimiento

La documentación y el mantenimiento son aspectos críticos en el proceso de programación y en la vida útil de un software. La documentación es importante porque proporciona una referencia completa del diseño y la funcionalidad del software. Proporciona información sobre cómo funciona el programa, cómo está estructurado y cómo interactúa con otros componentes del sistema. Además, la documentación puede servir como una guía para aquellos que mantendrán el software en el futuro.

El mantenimiento, por otro lado, es crítico para asegurar que el software siga siendo útil y efectivo con el tiempo. El software se enfrenta a una variedad de desafíos a lo largo de su vida útil, incluyendo cambios en el hardware, actualizaciones del sistema operativo y la necesidad de adaptarse a nuevas necesidades del usuario. El mantenimiento asegura que el software siga funcionando como se espera, que se corrijan los errores y que se realicen mejoras necesarias.

En resumen, la documentación y el mantenimiento son esenciales para asegurar que el software sea fácil de entender y mantener con el tiempo. La documentación adecuada puede ayudar a los desarrolladores a entender rápidamente el funcionamiento del software y realizar cambios necesarios. El mantenimiento continuo asegura que el software siga siendo efectivo a medida que cambian las necesidades del usuario y el entorno tecnológico.

Programas

En el contexto de la informática y la programación, un programa se refiere a un conjunto de instrucciones escritas en un lenguaje de programación específico que se utilizan para realizar una tarea en una computadora u otro dispositivo electrónico. Los programas se componen de una serie de instrucciones que se ejecutan en una secuencia lógica y están diseñados para llevar a cabo tareas específicas, como la realización de cálculos, la manipulación de datos, la interacción con el usuario o la automatización de procesos.

Los programas pueden ser tan simples como una sola línea de código o tan complejos como un conjunto de miles de líneas de código que abarcan múltiples archivos. Algunos ejemplos comunes de programas incluyen aplicaciones de software, como procesadores de texto, hojas de cálculo, navegadores web y programas de edición de video. Los programas también pueden ser scripts o secuencias de comandos que se utilizan para automatizar tareas, como copiar archivos o realizar actualizaciones de software en una red de computadoras.

La creación de programas es una habilidad fundamental en la programación y la informática, y los programadores utilizan una variedad de herramientas y técnicas para desarrollar y depurar programas. Los programas pueden ser escritos en una amplia variedad de lenguajes de programación, y los programadores también utilizan herramientas de desarrollo integrado (IDE) para ayudarles a escribir, depurar y probar código. Una vez que se ha desarrollado un programa, es importante documentar su funcionalidad y mantenerlo actualizado para asegurarse de que siga funcionando correctamente en el futuro.

Existen diferentes tipos de programas según su finalidad y su estructura. A continuación, se describen algunos de los tipos más comunes:

1. Programas de aplicación: son los programas diseñados para realizar una tarea específica, como procesar textos, editar imágenes, navegar por internet, entre otros. Estos programas se ejecutan sobre un sistema operativo y suelen ser desarrollados por empresas o programadores independientes.
2. Programas de sistema: son los programas que forman parte del sistema operativo y permiten su correcto funcionamiento. Ejemplos de programas de sistema son los controladores de dispositivos, el gestor de memoria, el gestor de archivos, entre otros.
3. Programas de utilidad: son los programas que ayudan a realizar tareas específicas, como la copia de seguridad de datos, la eliminación de archivos innecesarios, la detección de virus, entre otros.
4. Programas de lenguaje de programación: son los programas que permiten escribir, probar y depurar programas en un lenguaje de programación específico. Ejemplos de programas de lenguaje de programación son Eclipse, Visual Studio, NetBeans, entre otros.
5. Programas de juegos: son los programas diseñados para entretener al usuario y pueden ser desarrollados por empresas de software especializadas en juegos o por programadores independientes.

En resumen, los diferentes tipos de programas existentes tienen finalidades diversas y están diseñados para resolver problemas específicos en diferentes contextos

Partes de un Programa

Un programa de computadora se compone de varias partes importantes que interactúan para realizar una tarea específica. Estas partes son:

- **Declaraciones:** son las instrucciones que se utilizan para definir y declarar variables y constantes en el programa.
- **Estructuras de control:** son las instrucciones que permiten controlar el flujo de ejecución del programa. Entre estas estructuras se encuentran las instrucciones de decisión, que permiten al programa tomar diferentes caminos en función de ciertas condiciones, y las instrucciones de bucle, que permiten repetir una sección de código un número determinado de veces.
- **Funciones y procedimientos:** son bloques de código que se pueden llamar varias veces en diferentes partes del programa para realizar una tarea específica. Las funciones devuelven un valor, mientras que los procedimientos no.
- **Entrada/salida:** son las instrucciones que permiten al programa recibir información del usuario a través del teclado, el mouse o cualquier otro dispositivo de entrada, y mostrar información en la pantalla o en un archivo de salida.
- **Comentarios:** son las líneas de texto que se utilizan para explicar lo que hace cada parte del programa. Los comentarios no se ejecutan y son útiles para que otros programadores entiendan el código.

Cada una de estas partes es importante para la construcción de un programa bien estructurado y eficiente. Es importante tener en cuenta que un programa bien escrito

no solo debe ser funcional, sino también fácil de leer, mantener y modificar en el futuro.

Elementos de la programación imperativa

La programación imperativa es un paradigma de programación que se enfoca en describir los pasos necesarios para resolver un problema, utilizando instrucciones específicas para modificar el estado de una computadora. Algunos de los elementos más importantes de la programación imperativa incluyen:

- **Variables:** Son espacios de memoria en los que se pueden almacenar valores. En la programación imperativa, se utilizan variables para guardar y manipular datos.
- **Instrucciones:** Son las órdenes que indican a la computadora qué acción debe realizar. Estas instrucciones pueden ser de diferentes tipos, como asignaciones, condicionales, bucles, entre otras.
- **Funciones:** Son bloques de código que se utilizan para realizar una tarea específica. En la programación imperativa, las funciones son una herramienta muy importante para organizar y reutilizar el código.
- **Estructuras de control de flujo:** Son las instrucciones que permiten controlar el orden en que se ejecutan las diferentes partes de un programa. Algunas de las estructuras de control de flujo más comunes incluyen los condicionales (if-else), los bucles (for, while), y las instrucciones de salto (goto).
- **Programación orientada a objetos:** Es un enfoque de programación que se basa en la creación de objetos, que son instancias de una clase. En la programación orientada a objetos, se utilizan conceptos como la herencia, el polimorfismo y la encapsulación para organizar y reutilizar el código.

En resumen, los elementos de la programación imperativa incluyen variables, instrucciones, funciones, estructuras de control de flujo y programación orientada a objetos, entre otros. Estos elementos son esenciales para describir los pasos necesarios para resolver un problema y permiten que el programador tenga un control preciso sobre el comportamiento de la computadora

Fases de Desarrollo de un Programa

El desarrollo de un programa consta de diferentes fases o etapas que se suelen seguir de manera secuencial. Estas fases pueden variar según la metodología de desarrollo que se utilice, pero en general suelen incluir las siguientes:

1. **Análisis de requisitos:** en esta fase se define qué es lo que se quiere conseguir con el programa y cuáles son las necesidades que debe satisfacer. Se identifican los requisitos funcionales (lo que debe hacer el programa) y no funcionales (cómo debe hacerlo).
2. **Diseño:** en esta fase se elabora la arquitectura del programa y se definen los módulos que lo compondrán, así como las interfaces entre ellos. También se elabora el diseño detallado de cada módulo.
3. **Codificación:** en esta fase se escribe el código fuente del programa, siguiendo el diseño previamente definido. Se utilizan los lenguajes de programación y las herramientas de desarrollo correspondientes.
4. **Pruebas:** una vez que se ha codificado el programa, se realizan pruebas para comprobar que funciona correctamente y que cumple con los requisitos definidos. Se pueden realizar pruebas unitarias (sobre cada módulo por separado) y pruebas de integración (sobre el programa completo).

5. Documentación: en esta fase se elabora la documentación del programa, que puede incluir manuales de usuario, manuales técnicos, documentación del código fuente, etc.
6. Mantenimiento: una vez que el programa está en producción, se realizan tareas de mantenimiento para corregir errores, mejorar el rendimiento, añadir nuevas funcionalidades, etc.

Fundamentos del entorno de desarrollo del lenguaje de programación

El entorno de desarrollo de un lenguaje de programación es el conjunto de herramientas y recursos que se utilizan para escribir, probar y depurar programas en ese lenguaje. Estos recursos pueden variar dependiendo del lenguaje y del entorno específico que se esté utilizando, pero algunos de los fundamentos comunes incluyen:

1. Editor de código: Es el programa que se utiliza para escribir y editar el código fuente del programa. Puede ser un editor de texto plano o un IDE (Entorno de Desarrollo Integrado) que incluya características adicionales como resaltado de sintaxis, autocompletado y depuración integrada.
2. Compilador o intérprete: Es el programa que se encarga de convertir el código fuente escrito por el programador en código ejecutable por la máquina. Los lenguajes de programación de bajo nivel, como el lenguaje de máquina, no requieren un compilador o intérprete porque el código se ejecuta directamente en la máquina. Los lenguajes de alto nivel, por otro lado, necesitan un programa que traduzca el código a un formato que pueda ser entendido por la máquina.

3. Depurador: Es una herramienta que permite al programador ejecutar el programa paso a paso, detener la ejecución en cualquier momento y examinar el estado de las variables, la pila de llamadas y otros aspectos del programa que pueden estar causando errores.
4. Bibliotecas: Son conjuntos de código preescrito que se pueden incluir en el programa para realizar tareas específicas, como operaciones matemáticas, acceso a bases de datos, interacción con el sistema operativo, entre otros.
5. Herramientas de control de versiones: Permiten al programador trabajar en equipo y mantener un registro de los cambios realizados en el código fuente. Los sistemas de control de versiones más comunes son Git y SVN.
6. Documentación y tutoriales: Una documentación adecuada y fácil de entender puede ser muy útil para un programador que está aprendiendo un nuevo lenguaje o trabajando en un nuevo proyecto. Los tutoriales también pueden ser una herramienta valiosa para ayudar a los programadores a aprender el lenguaje de programación y su entorno de desarrollo.

Bibliografía y Webgrafía

Patterson, D. A., & Hennessy, J. L. (2013). Computer organization and design: The hardware/software interface. Morgan Kaufmann.

Martínez, F., & Calvo-Manzano, J. A. (2016). Ingeniería del software: Fundamentos, métodos y estándares. Pearson

Garrido, A. (2015). Lenguajes de programación. Alfaomega

Fernández Calvo, R. (2006). Historia de los lenguajes de programación. Madrid: RA-MA Editorial.

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). Compiladores: principios, técnicas y herramientas (2a ed.). Pearson.

López, L. F. (2018). Programación en C: conceptos básicos. McGraw-Hill Interamericana de España.

Joyanes, L., & Nawaz, Z. (2017). Programación en C. Fundamentos y aplicaciones (4a ed.). McGraw Hill.

Pressman, R. S., & Maxim, B. R. (2015). Ingeniería del software: Un enfoque práctico. McGraw-Hill Education

Joyanes, L. y Fernández, M. (2012). Fundamentos de programación con Java. McGraw-Hill.

Glosario

Actualización: Proceso de modificar o corregir un programa para que siga funcionando correctamente en el futuro.

Análisis de datos: proceso de examinar datos para obtener información útil.

Análisis: Actividad de analizar los requerimientos del software y definir soluciones apropiadas.

Aplicaciones móviles: programas que se ejecutan en dispositivos móviles.

Aplicaciones web: programas que se ejecutan en un navegador web.

Aprendizaje automático: campo de la inteligencia artificial que se ocupa de enseñar a las computadoras a aprender de los datos.

Arquitectura del programa: Es la estructura general del programa, incluyendo los módulos y las interfaces entre ellos.

Automatización de procesos: Proceso de automatizar una tarea en un programa, para que esta se realice de forma autónoma.

Bibliotecas: Son conjuntos de código preescrito que se pueden incluir en el programa para realizar tareas específicas.

Cálculos: Operaciones matemáticas que se realizan en un programa para obtener un resultado.

Código fuente: Texto escrito en un lenguaje de programación que describe las instrucciones necesarias para que una máquina realice una tarea específica.

Comentarios: Líneas de texto que se utilizan para explicar lo que hace cada parte del programa. Los comentarios no se ejecutan y son útiles para que otros programadores entiendan el código.

Compiladores: Son programas que traducen todo el código fuente escrito por un programador a lenguaje de máquina en un solo paso. Toman el código fuente completo y lo procesan para generar un archivo ejecutable que se puede ejecutar en una computadora sin necesidad de tener el código fuente original.

Componentes del software: Las diferentes partes que conforman el software final, como módulos, funciones y clases.

Computadora: dispositivo electrónico que puede recibir, almacenar y procesar información.

Controladores de dispositivos: Programas que permiten la comunicación entre un dispositivo y un sistema operativo.

Declaraciones: Instrucciones utilizadas para definir y declarar variables y constantes en un programa.

Depuración: proceso de encontrar y corregir errores después de que se haya detectado un error, utilizando herramientas especiales, como depuradores, para

examinar el estado de las variables y las estructuras de datos en tiempo de ejecución.

Depurador: Es una herramienta que permite al programador examinar el estado de las variables, la pila de llamadas y otros aspectos del programa que pueden estar causando errores.

Desarrollo web: proceso de creación de sitios web y aplicaciones web.

Diseño: Actividad de definir cómo se deben estructurar los componentes del software para satisfacer los requerimientos.

Documentación y tutoriales: Son recursos que pueden ser muy útiles para un programador que está aprendiendo un nuevo lenguaje o trabajando en un nuevo proyecto.

Documentación: Actividad de crear documentación que describe el software, su diseño y su uso.

Documentación: proceso de proporcionar una referencia completa del diseño y la funcionalidad del software para facilitar la comprensión del mismo y servir como una guía para aquellos que mantendrán el software en el futuro.

Editor de código: Es el programa que se utiliza para escribir y editar el código fuente del programa.

Eficiencia en términos de costos y tiempo: El logro de los objetivos del proyecto de software con la menor cantidad de recursos y tiempo posible.

Entorno de desarrollo: Es el conjunto de herramientas y recursos que se utilizan para escribir, probar y depurar programas en un lenguaje de programación específico.

Entrada/salida: Instrucciones que permiten al programa recibir información del usuario a través del teclado, el mouse u otro dispositivo de entrada, y mostrar información en la pantalla o en un archivo de salida.

Error: falla en el código de programación que impide que el programa se ejecute correctamente.

Errores de sintaxis: errores que ocurren cuando el código no sigue las reglas de sintaxis del lenguaje de programación y se pueden identificar fácilmente mediante un mensaje de error del compilador o intérprete.

Errores de tiempo de ejecución: errores que ocurren cuando el programa se está ejecutando y algo inesperado sucede, como una división por cero o un archivo que no se puede encontrar.

Errores lógicos: errores que no se detectan mediante la verificación de la sintaxis del código o al ejecutar el programa, pero que hacen que el programa no produzca los resultados esperados.

Estructuras de control de flujo: Instrucciones que permiten controlar el orden en que se ejecutan las diferentes partes de un programa, como los condicionales, los bucles y las instrucciones de salto.

Estructuras de control: Instrucciones que permiten controlar el flujo de ejecución del programa, como las instrucciones de decisión y las instrucciones de bucle.

Funciones y procedimientos: Bloques de código que se pueden llamar varias veces en diferentes partes del programa para realizar una tarea específica. Las funciones devuelven un valor, mientras que los procedimientos no.

Gestión de contenidos: proceso de crear, publicar y administrar contenido en línea.

Gestión del proyecto: Actividad de coordinar y gestionar los recursos necesarios para el desarrollo del software.

Gestor de archivos: Programa que administra la creación, eliminación y modificación de archivos en un sistema operativo.

Gestor de memoria: Programa que administra el uso de la memoria en un sistema operativo.

Hardware: componentes físicos de una computadora.

Herramientas de control de versiones: Permiten al programador trabajar en equipo y mantener un registro de los cambios realizados en el código fuente.

IDE (Herramienta de Desarrollo Integrado): Aplicación de software que proporciona un conjunto de herramientas para el desarrollo de programas.

Ingeniería de software: Proceso de diseñar, crear, probar y mantener software de calidad.

Instrucción: una acción específica que se le indica a la computadora a través de un lenguaje de programación.

Instrucciones binarias: secuencias de ceros y unos que representan operaciones muy básicas que se realizan directamente en el hardware del ordenador.

Interacción con el usuario: Proceso de comunicación entre un usuario y un programa, donde el usuario introduce información y el programa proporciona una respuesta.

Interfaces: Son los puntos de comunicación entre los módulos del programa.

Interpretado: tipo de lenguaje de programación en el que el código fuente se traduce en tiempo real y se ejecuta línea por línea.

Intérpretes: Son programas que ejecutan el código fuente escrito por el programador línea por línea. Leen el código fuente y lo traducen a lenguaje de máquina a medida que se va ejecutando. No generan un archivo ejecutable, sino que interpretan el código fuente en tiempo real.

Lenguaje de Alto nivel: Son lenguajes de programación que tienen un alto nivel de abstracción, lo que significa que utilizan una sintaxis más cercana al lenguaje natural y menos cercana al lenguaje de máquina. Estos lenguajes permiten a los programadores escribir código de forma más rápida y sencilla, ya que se enfocan en la solución de problemas en lugar de preocuparse por la arquitectura de la máquina.

Ejemplos de lenguajes de alto nivel incluyen Python, Java, Ruby, C#, y JavaScript.

Lenguaje de Bajo nivel: Es un tipo de lenguaje de programación que está más cerca del lenguaje de máquina y que está diseñado para interactuar directamente con el hardware del sistema, como la CPU, la memoria y los dispositivos de entrada/salida. Utiliza un conjunto limitado de instrucciones básicas que se representan mediante códigos de dos o tres letras, a menudo en código hexadecimal o ensamblador. Ejemplos incluyen el lenguaje ensamblador de la CPU de Intel (x86), el lenguaje ensamblador de la CPU de ARM y el lenguaje de programación C.

Lenguaje de máquina: lenguaje que entienden los ordenadores, formado por instrucciones binarias.

Lenguaje de programación de bajo nivel: lenguaje que se acerca más al lenguaje de máquina y se utiliza para tareas de bajo nivel.

Lenguaje de programación: conjunto de símbolos, reglas y estructuras que permiten a los programadores comunicar instrucciones a una computadora u otro tipo de dispositivo.

Lenguaje de scripting: lenguaje de programación utilizado para la automatización de tareas y procesos en una computadora.

Manipulación de datos: Proceso de modificar o procesar información en un programa.

Mantenimiento: proceso crítico para asegurar que el software siga siendo útil y efectivo con el tiempo, corrigiendo errores y realizando mejoras necesarias para adaptarse a cambios en el hardware, actualizaciones del sistema operativo y las necesidades del usuario.

Módulo: Es una unidad lógica y funcional del programa que realiza una tarea específica.

Planificación: Actividad de definir los objetivos, alcance, entregables, y recursos necesarios para el proyecto de software.

Portátil: capacidad de un programa para funcionar en diferentes sistemas operativos.

Procesador: unidad central de procesamiento de una computadora que realiza operaciones y procesa datos.

Programa: Conjunto de instrucciones escritas en un lenguaje de programación específico que se utilizan para realizar una tarea en una computadora u otro dispositivo electrónico.

Programación orientada a objetos: Enfoque de programación que se basa en la creación de objetos, que son instancias de una clase, y que utiliza conceptos como la herencia, el polimorfismo y la encapsulación para organizar y reutilizar el código.

Programación: proceso de crear software utilizando un lenguaje de programación para instruir a una computadora.

Programador: persona que escribe programas de computadora.

Programas compiladores: programas que traducen el código fuente escrito en un lenguaje de programación a lenguaje de máquina para que pueda ser ejecutado.

Programas de aplicación: Programas diseñados para realizar una tarea específica, como procesar textos, editar imágenes, navegar por internet, entre otros.

Programas de juegos: Programas diseñados para entretener al usuario y pueden ser desarrollados por empresas de software especializadas en juegos o por programadores independientes.

Programas de lenguaje de programación: Programas que permiten escribir, probar y depurar programas en un lenguaje de programación específico.

Programas de sistema: Programas que forman parte del sistema operativo y permiten su correcto funcionamiento.

Programas de utilidad: Programas que ayudan a realizar tareas específicas, como la copia de seguridad de datos, la eliminación de archivos innecesarios, la detección de virus, entre otros.

Programas: Conjuntos de instrucciones y datos que se utilizan para controlar el comportamiento de las máquinas.

Pruebas de integración: Son pruebas que se realizan sobre el programa completo para comprobar que todos los módulos funcionan correctamente juntos.

Pruebas unitarias: Son pruebas que se realizan sobre cada módulo del programa por separado para comprobar su correcto funcionamiento.

Requisitos funcionales: Son las necesidades que debe satisfacer el programa, es decir, las funcionalidades que debe realizar.

Requisitos no funcionales: Son las necesidades que debe satisfacer el programa en cuanto a cómo realizar las funcionalidades, como pueden ser la seguridad, la eficiencia, la escalabilidad, entre otros.

Secuencia lógica: Orden lógico en el que se deben ejecutar las instrucciones de un programa para que este funcione correctamente.

Sistemas operativos: software que controla el hardware de una computadora y permite que los programas se ejecuten en ella.

Software de sistemas: programas que interactúan con el hardware de una computadora para permitir que otros programas se ejecuten.

Variables: Espacios de memoria en los que se pueden almacenar valores utilizados para guardar y manipular datos en la programación imperativa.

Verificación: proceso de revisión del código en busca de errores antes de ejecutar el programa mediante pruebas y revisiones cuidadosas del código para asegurarse de que las funciones se comporten según lo previsto y que no haya errores sintácticos o semánticos.